CS683 PA1: Task1

Team: Huffman

Shaik Awez Mehtab (23b1080), Satyam Sinoliya (23b0958),
Gautam Siddarth (23b0957), Mohana Evuri (23b1017)

Task 1: 2D Convolution

Setup

- Intel Core i5-12450H (4 P-cores + 4 E-cores). L1d 48 KiB/core, L2 1.25 MiB/P-core, L3 12 MiB shared.
- Hardware prefetchers are **disabled**.
- Everything pinned with `taskset -c 2`, so the process cannot migrate between a P-core and an E-core mid-run.
- `./bin/conv` is the given harness, used for the graded result.
- `bench/` is a report-only harness that runs exactly one kernel with no correctness pass, so `perf` counters belong to that kernel alone. All comparison tables and plots come from it, best-of-7, with stages interleaved. It also holds the 128-bit and untiled SIMD versions needed for 1C and 1D; the five graded files are untouched by them.
- Absolute times drift by up to 2x with machine temperature, so only compare numbers within one table.

Task 1A: Loop reordering and unrolling

What is changed

- **Reordering.** The naive version goes over the $W * H$ image only once, computing each pixel using the whole kernel. With the reordering we go over the whole $W * H$ image K^2 times, only adding the effect of one kernel element to the output. This requires the output to be zeroed first.
- **Unrolling.** On top of reordering, unrolled the innermost `ox` loop by k, checking for different k values, as described further. This was done by manually writing k statements for computing `o[ox + 0],...,o[ox + k - 1]` inside the loop.

1. What motivated the changes?

- The naive version fetches a kernel element for every tap of every output pixel. These always hit L1, but each is still a load instruction plus its address arithmetic.
- After reordering the weight is constant for a whole pass, so it stays in a register and those instructions disappear.
- Unrolling then removes the loop overhead that is left, which is a large fraction of the instruction stream when the body is only 4 instructions.

2. What considerations did I take into account?

- Kept `ker[ky * K + kx]` for each outer two loop iteration in a variable, so this is stored in a register instead. If we instead used it directly, the compiler would not necessarily do that (replace that with something stored in a register), because it can't detect another pointer with same value could've modified the underlying value (eg. out). So it reads `ker` each inner iteration, which we're eliminating.

- `ox` is the innermost loop, so consecutive iterations touch consecutive addresses, which is the best case for the cache.
- **Unroll factor:** As mentioned before, tried various unroll factors, for which got these as time taken (in ms), for the corresponding params (K=3):

factor	256	512	1024	2048
1	0.365	1.489	6.306	~41
2	0.341	1.409	6.131	~41
4	0.321	1.368	5.829	~41
8	0.326	1.357	5.972	~41
16	0.312	1.363	5.793	~41

  - Loop overhead is ~2 instructions per iteration, so its share falls as 1/factor: 33% at 1, 11% at 4, 5.5% at 8. Most of the gain is already there by 4, and 4 vs 8 is within noise.
  - `W` is only guaranteed to be a multiple of 8, so 8 is the largest factor that divides every legal `W`. We used 8.
  - At 16 the body overruns the row on legal inputs like `W=8` and `W=24`, and the program segfaults on both. Testing further isn't that worthy, as the overhead is anyways 1/factor now.

3. How effective are the changes, and by which metric?

- The main metric we considered is wall-clock speedup over naive.
- Also counted **bytes moved past L2**, with `perf stat -e l2_rqsts.miss`. This overestimates DRAM traffic, since some of it is served by L3, but L3 misses would underestimate it by more.

Speedup over naive:

size	working set	reorder K=3	unroll K=3	reorder K=5	unroll K=5
128	128 KiB	1.26x	1.46x	1.30x	1.50x
256	0.5 MiB	1.23x	1.44x	1.31x	1.42x
512	2 MiB	1.26x	1.29x	1.24x	1.29x
1024	8 MiB	0.99x	1.03x	1.09x	1.13x
2048	32 MiB	0.61x	0.65x	0.65x	0.71x

So both changes help up to 512, break even at 1024, and are a clear **loss** at 2048.

Small sizes: Everything fits in cache, so nothing is waiting on memory and the loop is limited by instruction count. Measured with `perf` at 256x256:

version	instructions/pixel (rough)	IPC	cycles/pixel (rough)
naive	100.6	4.76	21.2
reorder	64.0	3.80	16.8
unroll	41.5	2.57	16.1

- Naive runs at IPC 4.76, close to the core's maximum of about 6 instructions per cycle, so it isn't waiting on anything, it's just executing too many instructions.
- Reordering cuts instructions per pixel by 36% and unrolling by a further 35%.
- For reordering that's the fewer loads, since we pre-store `ker[...]`. For unrolling it's the fewer loop update and check instructions.
- IPC also drops (4.76 to 3.80), since removing independent loads removes some parallelism, but instructions fall faster, so cycles/pixel still reduces.

Large sizes regress because reordering trades instruction count for memory traffic. The K^2 passes each re-read the input and do a read-modify-write of the output, so the traffic should be about $K^2 = 9$ times the compulsory minimum. Measured at 2048x2048, K=3, against a compulsory 32 MiB (16 MiB in read once, 16 MiB out written once):

version	L2 traffic	vs compulsory	time	effective rate
naive	40.7 MiB	1.27x	17.4 ms	2.5 GB/s
reorder	301.6 MiB	9.43x	28.4 ms	11.1 GB/s
unroll	301.6 MiB	9.42x	26.7 ms	11.9 GB/s

- The measured 9.43x matches the predicted 9 passes.
- Naive's 2.5 GB/s is nowhere near any memory limit, so it was never memory-bound to begin with, only instruction-bound.
- Reordering fixes the instruction count and immediately runs into the memory system instead.
- Whether the 9x traffic actually costs time depends on which cache level serves it. Up to 512 it is served by L2/L3 and is nearly free, so the instruction saving dominates. At 1024 (8 MiB) it is served by L3 and roughly cancels the win. At 2048 (32 MiB) it must come from DRAM and hence the loop inverts to 0.61x.
- **Unroll vs reorder at 2048** differ by only 3%. `perf` shows: going from unroll factor 4 to 8 removes 3.2% of instructions (5.997e9 to 5.808e9) but leaves cycles unchanged (1.6968e9 to 1.6925e9), this is because the core is waiting on memory.
- The same unrolling is worth 10-16% at 256 and 512, where instructions are the bottleneck.

Conclusion. Both changes are right for the instruction count and wrong for the memory system once the working set leaves cache. So we don't want to undo them, we want the K^2 passes to hit cache instead, which is what tiling does next.

Task 1B: Tiling

1. Profiling the baseline

- L1-D cache size on this machine: **48 KiB per core** (`lscpu` reports 320 KiB over 8 instances; the 4 P-cores have 48 KiB each).
- Baseline naive L1-D MPKI, from `perf stat -e instructions,L1-dcache-load-misses` (median of 3 runs):

naive L1-D MPKI	256	512	1024	2048
K=3	0.95	0.38	0.20	0.12
K=5	0.47	0.18	0.10	0.07
K=7	0.29	0.12	0.06	0.04
K=9	0.20	0.10	0.06	0.05

- MPKI falls with K only because instructions per pixel grows as K^2 (104, 218, 380, 590 for K=3,5,7,9) while the miss count barely moves. K does not change the picture; the rest of 1B uses K=3.

2. Tiled implementation

- `conv_tile.cpp` processes the output in $TH * TW$ blocks. For each block it zeroes the block, then runs the K^2 passes over that block alone.
- The block and its corresponding $(TH + K - 1) * (TW + K - 1)$ input block stay in cache for the whole sweep, instead of being re-fetched K^2 times from DRAM.

Tile sizes swept at $2048 * 2048$, `K=3`. Footprint is output tile + input block size:

tile	footprint	time (ms)	L1-D MPKI	L2 traffic
16x64	8.6 KiB	13.28	16.73	40.8 MiB
32x128	33.3 KiB	14.06	17.25	41.9 MiB
64x64	33.0 KiB	14.68	18.23	43.8 MiB
64x128	65.5 KiB	13.82	17.21	45.2 MiB
64x256	**130.5 KiB**	**13.42**	**16.72**	**44.7 MiB**
128x256	259.0 KiB	15.30	16.90	87.1 MiB
128x512	517.0 KiB	15.25	16.62	113.9 MiB
256x512	1030.0 KiB	16.35	16.70	253.6 MiB

I kept **64x256**. 16x64 is 1% faster at 2048 but slower at 1024 (2.999 vs 2.819 ms), so 64x256 is the better all-round choice.

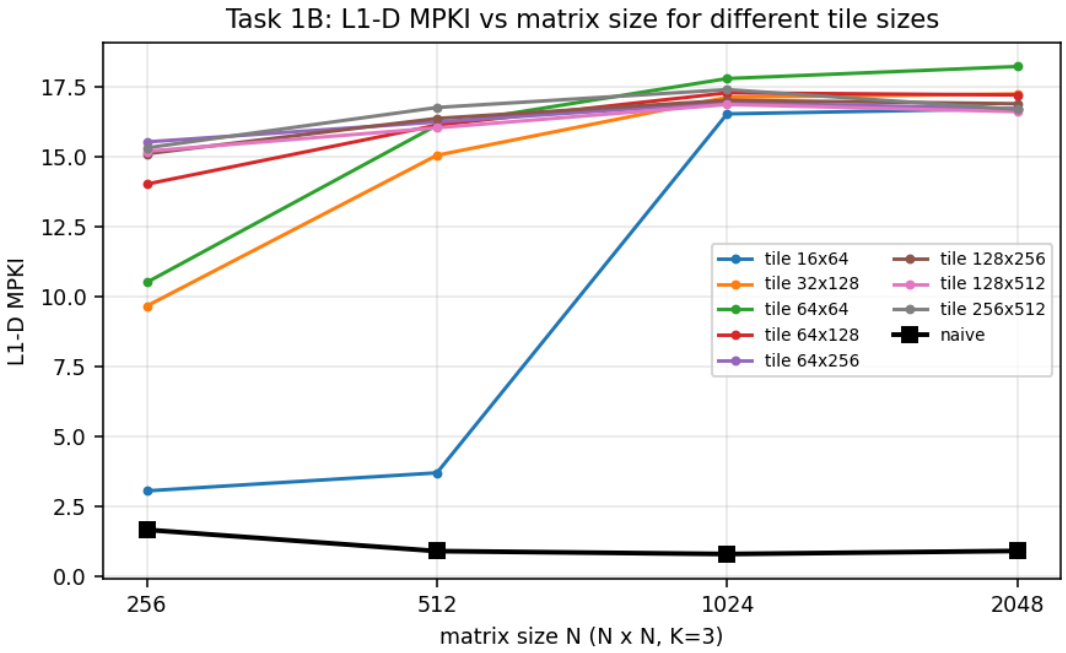


Figure 1: L1-D MPKI vs matrix size for different tile sizes. Every tiled curve sits far above naive, because tiling targets L2 rather than L1.

3. Analysis

Q1. Change in L1-D MPKI from naive to tiled.

- It goes **up**, from 0.12 to 16.8 at $2048 * 2048$, which looks wrong at first.
- MPKI is misses divided by instructions, and tiling changes both. Tiling is applied on top of the reordered loop, which already executes half as many instructions as naive (49 vs 104 per output pixel), so the denominator shrinks.
- The numerator grows too: naive re-reads its 3 active rows straight out of L1, whereas the tiled loop streams a 130 KiB tile that does not fit in a 48 KiB L1, so its reuse is served by **L2, not L1**.

The right comparison is tiled against the untiled reordered loop it replaces, and at the level tiling actually targets (i.e L2):

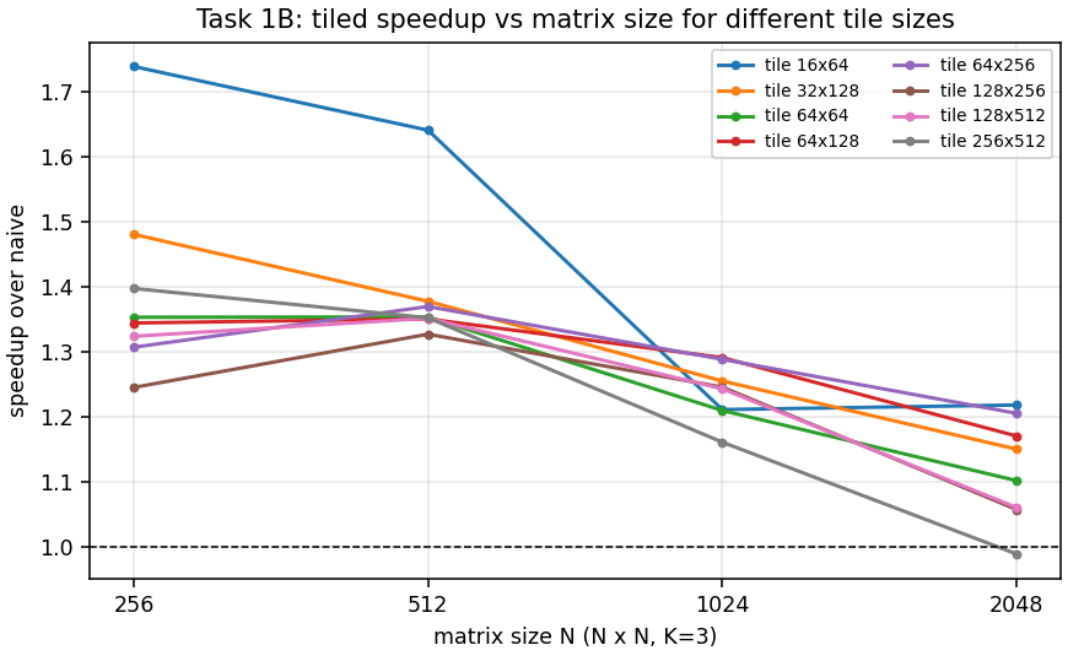


Figure 2: Tiled speedup vs matrix size for different tile sizes.

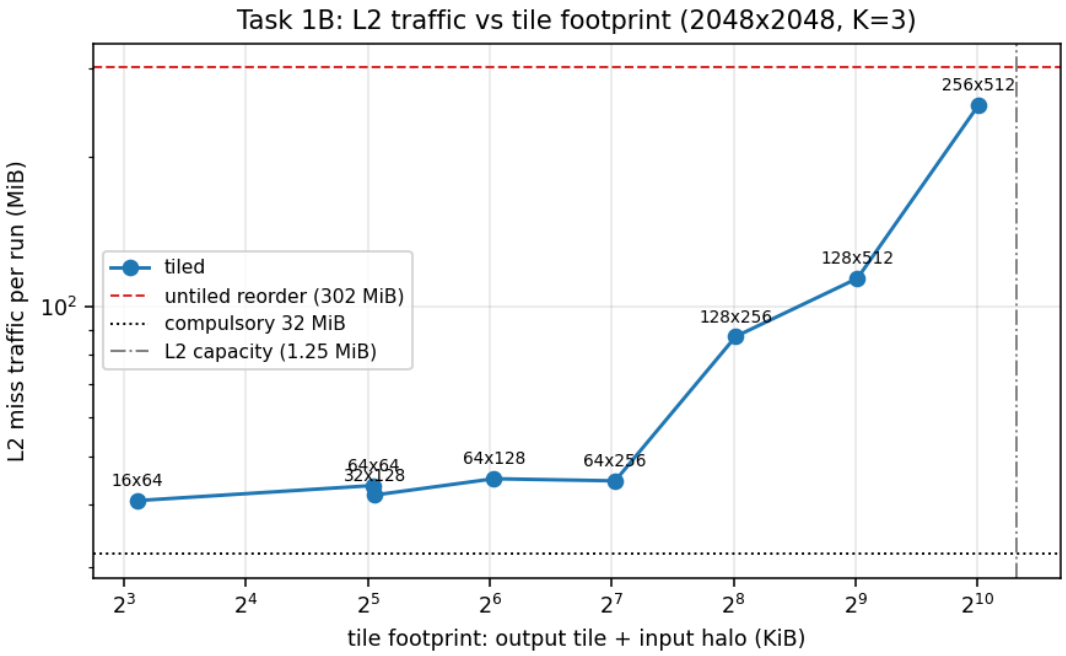


Figure 3: L2 traffic vs tile footprint at $2048 * 2048$. Traffic sits at the compulsory floor while the footprint stays well below L2 capacity, then climbs towards the untiled figure.

version	L2 traffic	vs compulsory	speedup vs naive
reorder (untiled)	301.6 MiB	9.43x	0.61x
tile	44.9 MiB	1.40x	1.17x

Tiling cuts traffic past L2 by **6.7x**, from 9.4 passes over the image down to 1.4, which is what we wanted. It was never about L1 MPKI.

Q2. How MPKI varies with matrix size and tile size.

- Fixing the size at 2048 and varying the tile, MPKI barely moves, it stays at 16.6-18.2. Every tile we tried is either bigger than the 48 KiB L1 or just streams through it, so none of them change what L1 sees.
- Varying the matrix size, the only tile that behaves differently is 16x64, whose 8.6 KiB footprint actually fits in L1: MPKI is 3.06 at 256 and 3.71 at 512, against 15-17 for every bigger tile.
- Even 16x64 goes back up to ~16.5 past 1024. The padded row stride is 2050 floats there, so the 18 halo rows it touches are ~8.2 KiB apart, and they start conflicting in a 12-way L1.
- L2 is where the working set argument actually shows up. Traffic sits at the ~41-45 MiB floor while the footprint is under ~130 KiB, then climbs: 87 MiB at 259 KiB, 114 MiB at 517 KiB, 254 MiB at 1030 KiB, which is almost the untiled 302 MiB.
- So the tile should be sized well **below** L2, not at it. At 80% of the 1.25 MiB L2 we've lost nearly all the benefit, because the tile has to survive K^2 passes while the streaming input is competing for the same cache.

Q3. Did I achieve a speedup?

Yes, and how it changes with size matters:

size	128	256	512	1024	2048
tile vs naive (K=3)	1.38x	1.40x	1.22x	1.18x	1.17x
tile vs naive (K=5)	1.42x	1.28x	1.34x	1.30x	1.33x
tile vs reorder (K=3)	1.10x	1.14x	0.97x	1.19x	1.91x

- Against naive the gain is modest, but against the reordered loop it is 1.91x at 2048, turning a 0.61x regression into a 1.17x speedup.
- What's left is that this stage is still scalar: 49 instructions per output pixel, and 2 loads + 1 store + 1 FMA for every single element. No amount of cache tuning fixes that, SIMD does.

Task 1C: SIMD

1. Baseline instruction count

Naive at 2048x2048, K=3: **0.436e9 instructions per run**, about 104 per output pixel.

2. Implementation

- `conv_simd.cpp` keeps the tiled nesting and replaces the scalar inner loop with 256-bit AVX2.
- `bench/conv_simd128.cpp` is the same code at 128-bit, for the width comparison.
- **AVX-512 is not available on this CPU**, so only 128-bit and 256-bit are reported.

3. Instruction count and performance

Per run at 2048x2048, K=3:

version	instructions	per output pixel	vs naive	speedup
naive	0.436e9	103.9	1.00x	1.00x
tile (scalar)	0.207e9	49.4	2.10x fewer	1.17x
simd 128-bit	0.104e9	24.7	4.21x fewer	3.06x
simd 256-bit	0.080e9	19.0	5.48x fewer	3.82x

- Going from naive to 256-bit SIMD, instructions drop **5.48x**. One `vfmadd` now does 8 elements where the scalar loop needed one multiply-add each, and the loop runs 8 times fewer iterations, so the loads, stores, increments and branches divide by 8 too.
- We don't get the full 8x because the zeroing pass, the tile bookkeeping and the loop setup are the same as before, they don't vectorise.
- Halving the width to 128-bit costs us 1.30x more instructions and 1.25x more time, which track each other closely. So we're still mostly limited by how many instructions we execute, and doubling the width buys us almost the full 2x.

4. Multi-size results

Speedup over naive, K=3:

size	128	256	512	1024	2048
128-bit	4.09x	4.01x	3.72x	3.17x	3.06x
256-bit	5.39x	5.27x	4.65x	4.38x	3.82x

Both widths lose ground as the matrix grows, and 256-bit loses more, so the gap narrows from 1.32x at N=128 to 1.25x at N=2048. Every size up to 1024 still fits in the 12 MiB L3, but at 2048 the input and output are 32 MiB together, so we're waiting on DRAM. Removing more instructions doesn't help then, and the wider version reaches that point first.

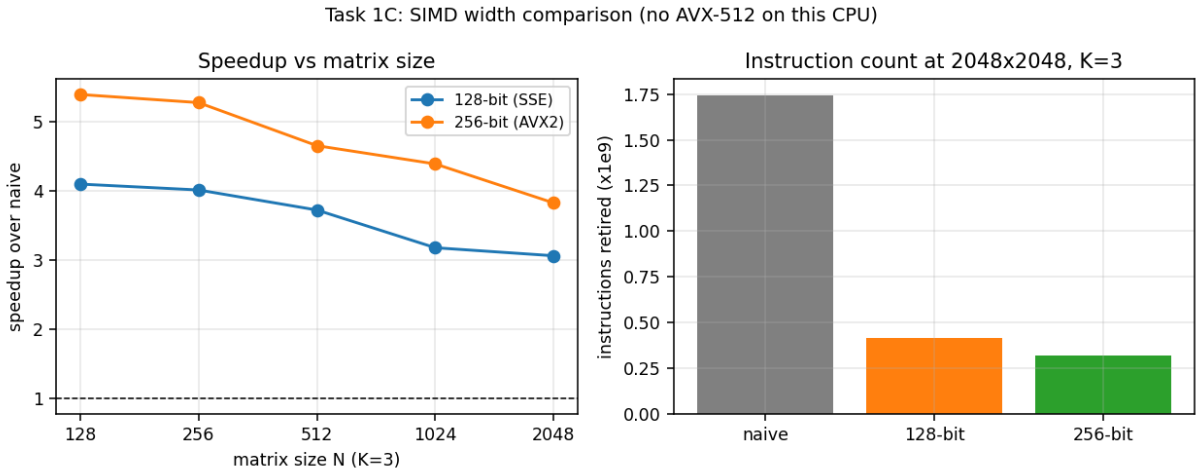


Figure 4: SIMD width comparison: speedup vs matrix size, and instruction count at 2048x2048.

5. Which intrinsics, and why

intrinsic	why
`_mm256_fmadd_ps`	one instruction for multiply-add, half the uops of separate mul+add and a single rounding step
`_mm256_set1_ps`	broadcasts the kernel weight once per (ky,kx) pass, so the FMA operand is already in a register
`_mm256_load_ps` / `_mm256_store_ps`	aligned access on `out`. `alloc_floats` uses `_mm_malloc(..., 64)` and `W % 8 == 0`, so every output row start is 32-byte aligned
`_mm256_loadu_ps`	unaligned load for `in`. The padded stride `W+2p` and the `+kx` shift both break alignment, so the aligned form would fault
`_mm256_setzero_ps`	vector zeroing of the output tile

The inner loop computes **two vectors per iteration**, so 16 columns instead of 8. This is the same unrolling as in 1A, just on the vector loop.

- We wrote the one-vector version too and measured both. The difference is within run-to-run noise: one vector is about 7% slower at 256, and 3-9% faster at 1024 and 2048.
- So doing two at a time isn't where the speedup comes from. Even with one, consecutive iterations write to different addresses, so the CPU already overlaps them on its own. We kept two only because it halves the loop bookkeeping per element.

Task 1D: Sabka saath sabka vikaas

`conv_optimized.cpp` combines everything, and adds a few things the earlier stages don't have:

- **Kernel loops innermost.** We accumulate the K^2 taps in vector registers and write each output vector once, so the read-modify-write of `out` is gone.
- **Non-temporal stores** (`_mm256_stream_ps`, with `_mm_sfence`), so we never fetch an output line before overwriting it.
- **Software prefetch** (`_mm_prefetch`) on the one input row that's actually new, two lines ahead.
- A K=3 version with all 9 weights sitting in registers.

Do the techniques compose?

At 2048x2048, K=3, each technique measured alone and together:

version	speedup
tiling alone (scalar)	1.17x
SIMD alone (untiled)	1.17x
tiling + SIMD	3.82x
all combined	17.04x

- Separately they give 1.17x each. Adding them would be ~1.34x and multiplying ~1.37x, but together we get **3.82x**, which is much more than either.
- That's because each one was hiding the other's benefit. Untiled SIMD gets instructions per pixel from 104 down to 18, but at 2048 it moves 9.86x the compulsory traffic, so it just waits on DRAM and the extra throughput has nowhere to go: 1.17x.
- Tiling gets traffic down to 1.40x compulsory, but the loop is still scalar: also 1.17x.
- With both, we're neither waiting on DRAM nor executing too many instructions, so the gains that were being masked finally show up.

The full version goes further still, and the traffic table shows why:

version	L2 traffic	vs compulsory	effective rate
naive	40.7 MiB	1.27x	2.5 GB/s
reorder	301.6 MiB	9.43x	11.1 GB/s
tile	44.9 MiB	1.40x	3.2 GB/s
simd (untiled)	315.7 MiB	9.86x	22.3 GB/s
simd (tiled)	44.0 MiB	1.38x	10.2 GB/s
optimized	28.8 MiB	0.90x	29.7 GB/s

- `optimized` moves **less than the compulsory 32 MiB**. The non-temporal stores keep the 16 MiB of output out of the cache entirely, so we never read a line just to overwrite it.
- It sustains 29.7 GB/s where the others sit near 10. That's the software prefetch doing what the disabled hardware prefetchers would have: starting the line fills early enough that misses overlap, instead of happening one after another.

Final results

Speedup over naive, K=3:

technique	128	256	512	1024	2048
loop reordering	1.26x	1.23x	1.26x	0.99x	0.61x
loop unrolling	1.46x	1.44x	1.29x	1.03x	0.65x
tiling	1.38x	1.40x	1.22x	1.18x	1.17x
SIMD (AVX2)	4.72x	5.42x	2.23x	1.60x	1.17x
tiling + SIMD	5.39x	5.27x	4.65x	4.38x	3.82x
all combined	18.06x	17.86x	19.63x	19.44x	17.04x

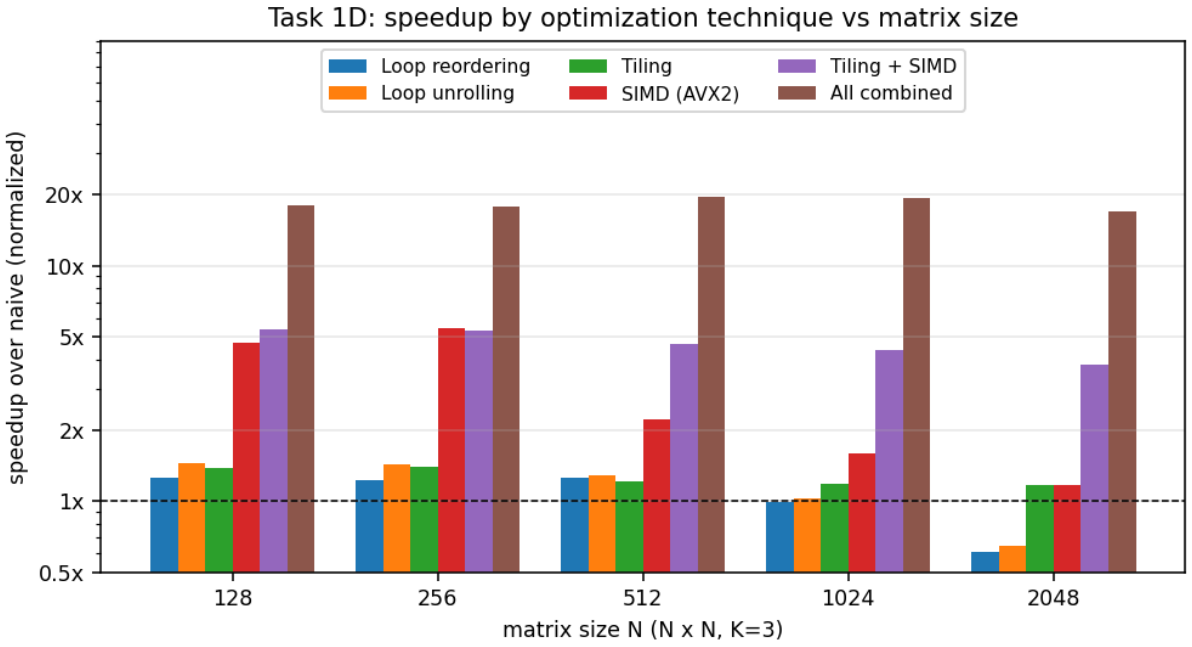


Figure 5: Speedup by optimization technique vs matrix size. Log y axis, since the techniques span 0.61x to 19.6x.

The same data as lines, which shows the trend with size more clearly:

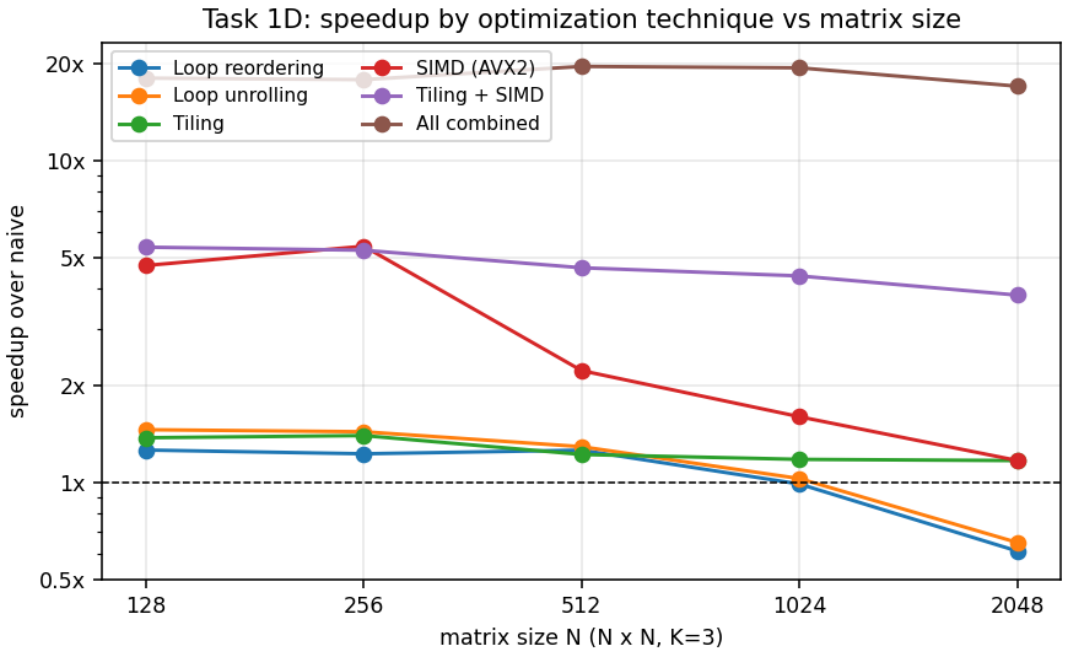


Figure 6: Speedup by optimization technique vs matrix size (log scale).

Varying kernel size at N=1024:

technique	K=3	K=5	K=7	K=9
loop reordering	0.99x	1.09x	0.91x	1.07x
loop unrolling	1.03x	1.13x	0.93x	1.05x
tiling	1.18x	1.30x	1.10x	1.32x
SIMD (AVX2)	1.60x	1.88x	1.51x	1.76x
tiling + SIMD	4.38x	5.60x	4.29x	4.83x
all combined	19.44x	19.87x	15.54x	16.28x

- The scalar stages stay within 0.9-1.3x at every K, so kernel size doesn't really matter to them. What they remove scales with K^2 just like the total work does.
- The vector stages gain a lot more.
- The combined one dips at K=7 and K=9, where we fall back to the generic `convK` and the weights no longer all stay in registers.

